函数调用时栈里发生了什么：栈帧与调用约定

嵌入式 Linux 手记　2026-10-28　公众号文章 · 原链接 mp.weixin.qq.com

先猜答案。这段代码在 Linux x86-64 上，参数 1、2、3 是怎么传给 `add3` 的?

```
int add3(int a, int b, int c) {
    return a + b + c;
}

int main(void) {
    int s = add3(1, 2, 3);
    return s;
}
```

老教材的口诀是"参数从右往左压栈"，很多人因此猜栈里压了三个数。答案是：一个都没压。`1`、`2`、`3` 分别装进 `edi`、`esi`、`edx` 三个寄存器直接送过去，栈上连一个字节的参数都没有。口诀没错，它属于 32 位的 cdecl 时代；64 位与 ARM 时代参数主要走寄存器，旧口诀不再适用。这一篇把一次函数调用的完整现场拆到指令级：跳进去之前发生什么、栈帧怎么搭、怎么准确跳回来。

一、调用约定：编译器之间的握手协议

先回答一个更基本的问题：参数放哪、返回值放哪、寄存器谁保存，这些事编译器自己定不行吗?

不行。你调用的 `printf` 不是你的编译器今天编出来的，它是 glibc 里编译好的二进制；你的代码将来还要被别人链接。参数从哪取、结果放哪、哪些寄存器用完要恢复原样，双方必须提前约好，这套约定就是调用约定（calling convention），它属于平台 ABI（应用二进制接口）的一部分。同一平台上，不同编译器、不同语言编出来的目标文件能互相调用，靠的就是都守这套约定。

主流平台的约定一张表说完：

平台	整型/指针参数	浮点参数	返回值
x86-64 System V（Linux/mac OS）	前 6 个走 rdi rsi rdx rcx r8 r9	xmm0~xmm7	rax

平台	整型/指针参数	浮点参数	返回值
Windows x64	前 4 个走 rcx rdx r8 r9	同槽位 xmm0~3	rax
arm32 AAPCS	前 4 个走 r0~r3	走 VFP 或栈	r0
arm64 AAPCS64	前 8 个走 x0~x7	v0~v7	x0

超出的参数才压栈。这也是为什么老口诀在今天会误导：寄存器够用时，栈只负责"溢出的那一部分"。

二、一次调用的完整现场：逐条指令回放

把 `add3(1, 2, 3)` 拆到指令级（x86-64，编译器开了优化的写法比这更短，这里取教科书形态）：

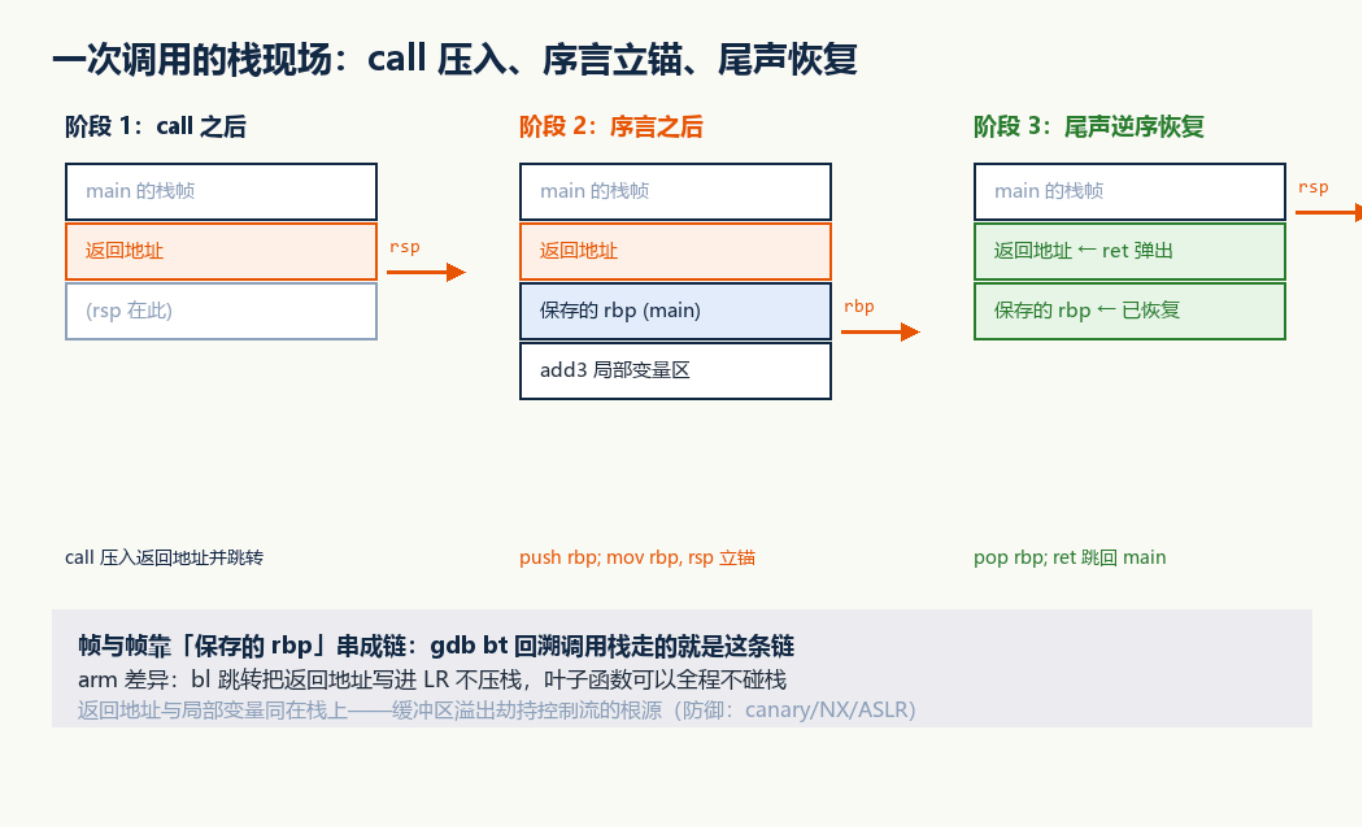
```
main 里:
    mov edi, 1          ; 第 1 个参数 → edi
    mov esi, 2          ; 第 2 个参数 → esi
    mov edx, 3          ; 第 3 个参数 → edx
    call add3           ; 压入返回地址，跳进 add3

add3 里:
    push rbp            ; 序言：保存调用者的帧指针
    mov rbp, rsp        ; 本帧的锚点立起来
    lea eax, [rdi+rsi]
    add eax, edx        ; 三数相加，结果进 eax
    pop rbp             ; 尾声：恢复调用者的帧指针
    ret                 ; 弹出返回地址，跳回 main
```

`call` 一条指令做了两件事：把返回地址（`call` 的下一条指令位置）压上栈，然后跳转。进入 `add3` 后先执行序言（prologue）：`push rbp` 保存调用者的帧指针，`mov rbp, rsp` 把栈指针的当前位置立为本帧的锚——此后本函数的局部变量全用 `rbp 减偏移` 来访问。返回前执行尾声（epilogue）逆序恢复：`pop rbp` 还回帧指针，`ret` 从栈顶弹出返回地址装进 `rip`，控制流精确回到 `main` 里 `call` 的下一条。

这个"锚"就是帧指针（frame pointer）。帧与帧之间靠"保存的 rbp"串成一条链，每一帧都记着上一帧的锚在哪：篇 17 的 `gdb bt` 能把调用链一层层打出来，回溯依据正是这条帧链（编译器若用 `-fomit-frame-pointer` 省掉 rbp，则靠 `.eh_frame` 表也能回溯，代价是信息来自表而非链）。

arm 平台有一个关键差异：`bl` 指令跳转时自动把返回地址写进链接寄存器 LR（arm64 是 `x30`），不压栈。叶子函数（不再调用别人的函数）从头到尾可以一次都不碰栈，比 x86 还省；只有非叶子函数才需要在序言里把 LR 压栈腾出 LR 另作他用。嵌入式读者看反汇编时注意：arm 的返回是 `bx lr` 或把压栈的 `lr` 弹回 `pc`，x86 的返回是 `ret` 从栈上取。



图：一次调用的栈现场。call 压入返回地址、序言立锚、尾声逆序恢复；帧与帧靠保存的 rbp 串成链，gdb bt 回溯走的就是它。

三、谁保存谁恢复：caller-saved 与 callee-saved

寄存器是全场共享的稀缺资源，`add3` 大摇大摆用 `edi`，就不怕踩坏 `main` 正在用的值？答案分两类。像 `rax`、`rcx`、`rdx` 这类登记为 caller-saved（调用方保存）的寄存器：约定允许被调方随便用、用完不恢复；调用方如果还想在调用之后继续用，得在 `call` 之前自己压栈保存。像 `rbx`、`r12~r15` 这类 callee-saved（被调方保存）寄存器则相反：被调方要用，就得先保存、退出前恢复，保证调用方看到的值分毫未动。

为什么这样分？各取所需。caller-saved 让叶子函数飞快：临时计算随手用 `rax`，一个字节都不用保存；callee-saved 让跨调用的长期变量可靠：循环计数器放 `rbx`，调用别人回来还在。arm32 的分工是 `r0~r3`、`r12` 归调用方保存，`r4~r11` 归被调方保存——读反汇编时看到 `pu`

`sh {r4, lr}` 开头的函数，就是在履行被调方义务：要用 `r4`，顺便把 `lr`（返回地址）也一起押上栈。



图：add3(1, 2, 3) 三平台传参对照与寄存器两班分工。参数走寄存器不进内存；caller-saved 让叶子函数快，callee-saved 让跨调用变量可靠。

初学者常见的误解在此顺带清一下："指针传参能改实参"并不违反这套约定。传址仍然是传值，复制的是地址值的副本，被调方凭副本去改被指向的对象；形参本身（那个副本）怎么改都影响不到主调变量。传参的机制自始至终只有一种：复制。

四、返回地址在栈上：便利与代价

x86 的设计把返回地址放在栈上，与局部变量同住一片区域，这带来一个延续至今的安全代价：缓冲区溢出可以覆盖返回地址，把控制流劫持到攻击者塞进来的地址（篇 49 的场景）。现代防御层层加码：栈保护（canary 金丝雀值）在帧里放哨，`NX` 位让栈页不可执行，篇 16 的 ASLR 让地址难以预测，机制层的根源都指向同一个设计决定。

还有一个对称设计值得记住：栈向低地址生长，堆向高地址生长，两者从内存段的两端相向而行，中间的空隙留给各自扩张。这是"为什么局部变量地址总比全局变量高"的答案。至此，`call` / `ret` 、帧链、寄存器分工、返回地址的去向全部就位，栈的账本已经摊开：但它到底有多大、什么时候会被写穿、RTOS 任务栈怎么算水位，是下一篇的事。

写在最后

1. 调用约定是平台 ABI 的握手协议：参数位置、返回值位置、寄存器保存责任，双方守约才能互相调用；主流平台参数优先走寄存器（x86-64 前 6 个 `rdi~r9`，arm64 前 8 个 `x0~x7`）。
2. `call` 压返回地址并跳转；序言保存帧指针立锚，尾声逆序恢复；`gdb bt` 回溯靠的正是帧链（或 `.eh_frame`）。
3. arm 的 `bl` 把返回地址写进 LR 不压栈，叶子函数可以全程不碰栈——这是它与 x86 `ret` 取栈的最大差异。
4. 寄存器分两班：caller-saved 随便用（调用方要留就自己先存），callee-saved 用前押后还（`rbx`、`r12~r15`；arm32 的 `r4~r11`）；传址仍是传值，副本机制与调用约定不冲突。
5. 返回地址与局部变量同在栈上，是缓冲区溢出劫持控制流的根源；栈向下堆向上相向而行，中间空隙各自扩张。

栈帧的账记完了，账本本身的极限还没量过：一个递归函数能把 8MB 的栈吃穿吗？RTOS 任务的栈为什么只有几百字节？下一篇《你的栈到底有多大：爆栈与任务栈计算》，把栈的测量、水位与防爆手段一次算清。

合集持续更新，往期与后续篇目都在目录里：C 语言底层实战 · 合集目录

后台回复「交流」，我很乐意分享我的学习计划和一线工作经验，直接来聊。